

Kapitel 6. Flödeskontroll

Innehållsförteckning

[if-satser](#)[if-else-satser](#)[Logiska operatorer](#)[switch-uttryck](#)[? :-operatorn](#)

Detta kapitel tittar närmare på de olika möjligheterna till *flödeskontroll* som finns i C++. Med flödeskontroll avses att ett program kan utföra olika satser på bas av ett villkors värde.

if-satser

Den enklaste former av flödeskontroll i C++ är en *if*-sats. Denna kontrollerar att villkor och om detta evalueras till `true` utförs *if*-satsens kropp, i annat fall ignoreras kroppen. Den allmänna formen ser ut så här:

```
if ( villkor )
    sats;
```

Även *if*-satser bör använda sig av kodblock för att undvika misstag. Ett exempel som kontrollerar om ett tal har ett visst värde kan se ut så här:

```
// ge en variabel Tal ett värde
int Tal;
cin >> Tal;
if ( Tal == 27 ) {
    // talet var 27
    cout << "Talet var 27!\n";
}
```

I C++ används inte nyckelordet `then` eller dylikt såsom i t.ex. Modula-2. Exemplet ovan skriver ut *Talet var 27!* om `Tal` har värdet 27.

if-satser är den mest använda konstruktionen för att styra ett programs flöde i C++. Så gott som alla program använder sig av *if*-satser. Man bör dock vara noggrann med att man faktiskt skriver `==` då man avser att göra en jämförelse. Vad händer om man istället hade skrivit:

```
if ( Tal = 27 ) {
    ....
}
```

Beroende på kompilatorn som används är detta helt korrekt C++ och inte ens en varning ges. Här har ett `=` fallit bort. Kvar blir då en tilldelning! Det som händer är att `Tal` tilldelas värdet 27 och `Tal` evalueras då till `true` eftersom det inte är 0, och *if*-satsens kropp kommer att utföras. Kanske inte direkt önskat resultat. Detta är ett vanligt fel, och kan vara svårt att hitta.

Gruppering av kod i block

Då man hanterar *if*-satser är det speciellt viktigt att tänka på att använda block runt *if*-satsen kropp. Speciellt om man avser att ha fler än en enda sats som kropp. Exemplet nedan fungerar inte som man kunde tro

```
// läs ett menyval från tangentbordet
```

```
char Menyval;
cin >> Menyval;
if ( Menyval == 's' )
    // spara någonting till fil
    cout << "Sparar data till fil." << endl;
    spara ();
```

Enligt indenteringen ovan så skulle funktionen `spara()` (mera om funktioner i [Kapitel 7](#)) utföras endast då menyvalet är ett `s`, men så är inte fallet. Endast raden med `cout` tillhör `if`-satsens kropp, raden med `spara()` kommer efter hela `if`-satsen och utförs således alltid. Kanske inte önskat beteende. Problemet åtgärdas genom att kapsla in kroppen mellan `{` och `}`:

```
// läs ett menyval från tangentbordet
char Menyval;
cin >> Menyval;
if ( Menyval == 's' ) {
    // spara någonting till fil
    cout << "Sparar data till fil." << endl;
    spara ();
}
```

Att använda block är något som är bra att göra till en vana. Koden blir minimalt längre, men antalet onödiga logiska fel minskar.

Nästlade `if`-satser

Det är fullt möjligt att nästla `if`-satser innanför varandra om det skulle behövas, precis som i andra programmeringsspråk. Detta kan se ut som nedan där ett tal först kontrolleras om det är jämnt, och sedan om det är delbart med 4 eller 6.

```
// ge en variabel Tal ett värde
int Tal;
cin >> Tal;
if ( Tal % 2 == 0 ) {
    // talet var jämnt, d.v.s. delbart med 2, är det delbart med 4?
    if ( Tal % 4 == 0 ) {
        // talet var delbart med 4
        cout << "Talet " << Tal << " är delbart med 4\n";
    }

    // är det delbart med 6?
    if ( Tal % 6 == 0 ) {
        // talet var delbart med 6
        cout << "Talet " << Tal << " är delbart med 6\n";
    }
}
```

[Förutgående](#)

Avbryta och fortsätta slingor

[Hem](#)

[Nästa](#)

`if-else-satser`